

Modern AutoCrit: A Modular GPT-Based Clinical-Trial Eligibility-Criteria Extractor

Jonathan Ma

August 20, 2026

Abstract

Modern AutoCrit is a modular modernization of AutoCriteria, a GPT-based system for extracting granular clinical-trial eligibility criteria from ClinicalTrials.gov documents. This report documents the current repository architecture and every Python function or method, explains the differences from the upstream inspiration, records key development milestones, and proposes a path toward an agentic eligibility-extraction system. The report describes software behavior; it does not assert that model outputs are clinical ground truth or suitable for autonomous eligibility decisions.

Contents

1	Purpose, scope, and lineage	3
2	System architecture	3
2.1	Layered view	3
2.2	Primary extraction flow	4
2.3	Integrity-validation flow	4
2.4	Data contracts	4
3	Complete active-code function reference	5
3.1	Entry points and application coordination	5
3.2	Core extraction, parsing, prompting, and normalization	5
3.3	Pipeline and quality control	7
3.4	Infrastructure services	8
3.5	Integrity validation agent	11
3.6	Tkinter presentation layer	12
4	Command-line data utilities	15
4.1	<code>data_download.py</code>	15
4.2	<code>dict_mapping.py</code>	15
5	Tests, benchmarks, backups, and placeholders	16
5.1	Test functions	16
5.2	Retained PySide backup	17
5.3	Empty compatibility and future-extension modules	17
6	What changed from AutoCriteria	17

7	Key milestones	18
8	Current limitations and technical debt	19
9	Future work: toward an agentic eligibility-extraction system	19
9.1	Terminology grounding with RxNorm and ICD-10	19
9.2	Embedding-assisted adaptive dictionaries	20
9.3	Reviewer/critic agent	20
9.4	Browser-based deployment	20
9.5	Proposed agentic loop	20
10	Recommended evaluation program	21

1 Purpose, scope, and lineage

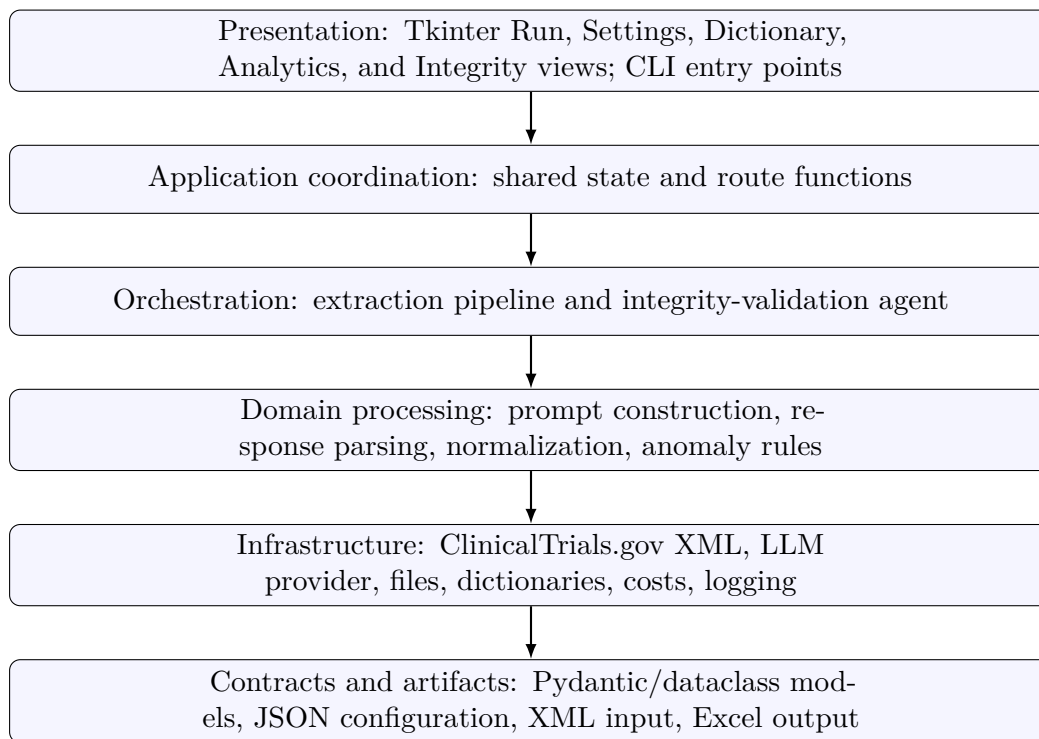
The project is inspired by Datta et al.’s AutoCriteria system and its public repository [1, 2]. The upstream program accepts ClinicalTrials.gov XML, separates inclusion and exclusion text, prompts GPT-4 for structured fields, performs substantial response cleanup, and writes tabular results. Its public implementation places prompt templates, LangChain calls, XML traversal, retry logic, post-processing, and dataframe construction in a single `eligibility_criteria_extraction.py` script [1].

Modern AutoCrit preserves the central scientific objective—turning free-text eligibility criteria into auditable structured rows—but changes the software shape. It separates domain models, parsing, prompting, model access, normalization, quality checks, orchestration, persistence, and presentation. It uses the modern OpenAI SDK through a provider abstraction instead of LangChain, supplies both command-line and standalone Tkinter workflows, and adds deterministic anomaly screening plus an independent LLM-assisted integrity audit.

This document covers all tracked Python source files in the current tree, including active runtime code, command-line utilities, tests, benchmark placeholders, compatibility placeholders, and the retained PySide backup. Generated data, JSON dictionary entries, images, and notebook cells are architectural assets but are not Python function definitions and therefore are summarized rather than enumerated as callables.

2 System architecture

2.1 Layered view



The active dependency direction is mostly downward: views call route/service objects; the pipeline composes extraction, normalization, quality analysis, and file output; services isolate external

systems; schemas define cross-layer contracts. The integrity agent is deliberately adjacent to, rather than inside, the extraction pipeline so it can audit outputs against source text independently.

2.2 Primary extraction flow

1. `ClinicalTrialsService` discovers XML files and loads trial metadata and eligibility text.
2. `ModernAutoCritExtractor` separates inclusion/exclusion sections, chunks text, builds a prompt, calls the configured LLM provider, and parses each response into `EligibilityCriterion` objects.
3. `CriteriaNormalizer` canonicalizes entity, attribute, disease, and value text; returns unmapped terms; and removes exact duplicates.
4. `AnomalyDetector` applies numeric, unit, terminology, and missing-value checks.
5. `ModernAutoCritPipeline` writes the criteria, unmapped-term, and anomaly Excel workbooks and returns a `PipelineSummary`.
6. The Run and Analytics views display progress and results without placing extraction logic in GUI widgets.

2.3 Integrity-validation flow

The Integrity view supplies source XML, an extraction workbook, optional trial IDs, and an output path. `IntegrityValidationAgent` aligns workbook rows with each trial, asks an LLM to assess source-clause coverage and row support, validates returned indices, assigns conservative defaults for omitted rows, calculates estimated metrics, and exports a multi-sheet workbook. Follow-up chat is grounded in a reduced JSON representation of the completed audit. These metrics are model-assisted estimates, not clinician-annotated performance measurements.

2.4 Data contracts

Type	Responsibility
<code>EligibilityCriterion</code>	One extracted row: trial/type, entity, attribute, value, temporal qualifier, modifier, source sentence, phase, and URL.
<code>ExtractionResult</code>	Criteria and non-fatal errors produced for one trial.
<code>PipelineSummary</code>	Counts before/after normalization and deduplication, cost, output paths, and anomaly counts by severity/category.
<code>LLMSettings</code>	Provider, credential, model, base URL, temperature, and output-token limit.
<code>AppSettings</code>	Aggregates extraction, cost, directory, and dictionary settings.
<code>AnomalyFinding</code>	Rule identifier, affected row/fields, category, severity, message, action, and confidence.
<code>ClauseValidation</code>	Source clause, coverage classification, matched rows, missing concepts, explanation, and confidence.
<code>ExtractionValidation</code>	Extracted-row support classification and its source evidence.
<code>ValidationMetrics</code>	Coverage/support counts and estimated precision, recall, F1, missing-value, and atomicity rates.
<code>TrialValidationResult</code> / <code>ValidationRunResult</code>	Per-trial findings and multi-trial run state, report path, errors, and cancellation status.

3 Complete active-code function reference

This section inventories every function, property, and method in active production modules. Constructors are included because they establish dependencies and runtime state. Private and nested helpers are included explicitly.

3.1 Entry points and application coordination

main.py

main()

Ensures runtime directories exist and launches the Tkinter application.

run_pipeline.py

main()

Builds default LLM, extraction, cost, normalizer, and pipeline objects; processes the configured XML directory; and prints the resulting summary for a GUI-free run.

backend/api/app_state.py

create_app_state()

Loads persisted settings and constructs shared cost-monitor and disabled Codex-bridge state for the GUI.

backend/api/routes.py

save_settings(state)

Persists the settings held in shared application state.

run_pipeline(state, xml_directory, output_excel)

Constructs the extractor, normalizer, and pipeline from state, runs extraction, and records the current output and last summary.

preview_output(output_excel, n)

Returns the first *n* rows of a saved output workbook.

load_dictionary(dictionary_type)

Loads a named normalization dictionary through DictionaryService.

save_dictionary(dictionary_type, mapping)

Persists a complete named mapping.

add_dictionary_mapping(dictionary_type, raw_term, canonical_term)

Adds or replaces one raw-to-canonical mapping.

3.2 Core extraction, parsing, prompting, and normalization

backend/core/extractor.py: ModernAutoCritExtractor

__init__(llm_settings, extraction_settings, cost_monitor)

Resolves defaults, retains extraction/cost configuration, and obtains an LLM provider from the factory.

extract_from_text(criteria_text, trial_id, criteria_type, phase, url, disease_context)

Word-chunks one inclusion or exclusion block, prompts the provider for every chunk, parses responses, and concatenates typed criteria.

extract_from_trial_dict(trial)
 Validates trial text, derives disease context, separates inclusion/exclusion sections, runs each independently, and returns criteria plus recoverable errors.

extract_from_xml_directory(xml_directory)
 Loads and extracts every XML trial in a directory, returning one `ExtractionResult` per file.

split_inclusion_exclusion(eligibility_text)
 Conservatively locates inclusion/exclusion headings and returns two text sections, falling back to treating unsplit text as inclusion.

backend/core/prompts.py

build_extraction_prompt(criteria_text, criteria_type, disease_context)
 Builds the provider-ready instruction for atomic structured extraction with source grounding and type-specific semantics.

build_temporal_prompt(sentence, attribute)
 Builds a focused prompt for obtaining the time constraint associated with an attribute; it remains available as a helper but is not on the primary extraction path.

backend/core/parser.py

extract_json_array(text)
 Finds and decodes a JSON array from raw or fenced model output, with bracket-boundary recovery.

is_bad_attribute(attribute)
 Rejects empty, placeholder, generic, or otherwise unusable attribute labels.

normalize_entity(entity)
 Maps common model variations to the repository's expected entity labels.

parse_extraction_response(response_text, trial_id, criteria_type, phase, url)
 Converts decoded model records into validated `EligibilityCriterion` objects while attaching trial metadata and filtering malformed rows.

backend/core/normalizer.py: CriteriaNormalizer

__init__(entity_map, attribute_map, disease_map)
 Loads supplied or repository dictionaries and prepares cleaned lookup keys.

_clean_map_keys(mapping)
 Whitespace- and case-normalizes dictionary keys for deterministic matching.

normalize_entity(entity)
 Maps an entity label to its canonical form while retaining a cleaned fallback.

normalize_attribute(attribute)
 Canonicalizes an attribute through the attribute and disease maps.

normalize_value(attribute, value)
 Cleans values and applies attribute-aware value normalization.

normalize_criterion(criterion)
 Normalizes all relevant fields in one typed criterion.

normalize_many(criteria)
 Normalizes a collection and converts it into the standard dataframe representation.

remove_exact_duplicates(df)
 Drops duplicate criteria using the domain fields that define row equivalence.

find_unmapped_terms(df, min_count)

Aggregates frequently observed entities/attributes absent from active dictionaries for reviewer attention.

find_fuzzy_duplicates(df, threshold)

Finds semantically similar candidate pairs using string similarity; this is an advisory helper rather than automatic deletion.

criteria_to_dataframe(criteria)

Serializes criteria into a dataframe with the canonical output-column order.

3.3 Pipeline and quality control

backend/services/pipeline.py: ModernAutoCritPipeline

__init__(extractor, normalizer, cost_monitor)

Injects the three principal pipeline collaborators.

run(xml_directory, output_excel)

Orchestrates trial loading, extraction, normalization, exact deduplication, anomaly analysis, unmapped-term analysis, workbook export, logging, and summary creation.

preview(output_excel, n)

Loads the first *n* rows of an Excel output.

backend/quality/anomaly_detector.py: AnomalyDetector

analyze_row(row, row_index)

Applies missing-value, numeric-range, unit, and terminology checks to one output row and returns structured findings.

analyze_dataframe(df)

Runs row analysis across a dataframe and concatenates all findings.

findings_to_dataframe(findings)

Serializes anomaly objects into a stable tabular report.

_none_if_missing(value)

Converts pandas missing values and empty values to *None* before rule evaluation.

backend/quality/rules/numeric_rules.py

extract_first_number(text)

Extracts the first signed integer or decimal from text.

extract_unit(text)

Recognizes and canonicalizes supported clinical measurement units.

extract_comparator(text)

Recognizes textual or symbolic comparison operators.

parse_measurement(text)

Returns a *ParsedMeasurement* containing number, unit, and comparator.

normalize_attribute_name(attribute)

Cleans an attribute key for rule lookup.

check_numeric_range(attribute, value)

Applies attribute-specific plausibility and dosage thresholds and returns rule findings. Its nested *add(...)* helper creates each finding consistently.

check_unit_compatibility(attribute, value)

Flags units incompatible with the recognized attribute family.

backend/quality/rules/terminology_rules.py

clean_term(term)

Normalizes whitespace and case for terminology checks.

check_bad_term(attribute, entity)

Detects placeholders, prompt leakage, overly generic labels, and malformed terms. Its nested `add(...)` helper records consistent findings.

check_value_term(value)

Detects missing, placeholder, or suspicious extracted values.

3.4 Infrastructure services

backend/services/clinicaltrials_service.py: ClinicalTrialsService

__init__(xml_directory)

Stores and validates the directory used for XML discovery.

xml_files()

Returns a deterministic list of XML files.

iter_trials()

Yields each path together with its parsed BeautifulSoup document.

nct_id(soup)

Extracts the trial identifier.

title(soup)

Extracts the brief title.

phase(soup)

Extracts the phase text.

condition(soup)

Extracts all condition labels.

url(nct_id)

Constructs the public ClinicalTrials.gov record URL.

eligibility_text(soup)

Extracts the free-text eligibility block.

load_trial(xml_path)

Parses one file and returns the trial dictionary consumed by the extractor.

backend/services/llm/base.py

BaseLLMProvider.generate(prompt, ...)

Abstract provider contract for text generation with model and sampling limits.

BaseLLMProvider.test_connection()

Sends a minimal deterministic request and verifies the expected response.

BaseLLMProvider.default_model

Abstract property defining a provider's default model name.

backend/services/llm/factory.py

create_llm_provider(settings, cost_monitor)

Selects a provider implementation from settings. The repository currently contains the OpenAI implementation; Anthropic, Gemini, and Ollama branches are extension points whose imported modules are not yet present.

backend/services/llm/openai_provider.py: OpenAIProvider
__init__(api_key, cost_monitor, max_retries)
 Resolves the API key, constructs the modern OpenAI client, and stores retry/cost collaborators.
default_model
 Returns the provider's default OpenAI model.
generate(prompt, ...)
 Calls the OpenAI Responses API, captures token usage, records cost, retries transient failures with exponential backoff, and gives quota exhaustion a targeted error.

backend/services/openai_client.py: legacy compatibility client
__init__(settings, cost_monitor, max_retries)
 Constructs the earlier OpenAI-specific client retained for compatibility.
generate_text(prompt, ...)
 Calls the Responses API with retries and returns output text.
generate_json(prompt, ...)
 Requests text and parses it as JSON.
_record_usage(response, model)
 Transfers token counts to the cost monitor.

backend/services/cost_monitor.py: CostMonitor
estimate_cost(model, input_tokens, output_tokens)
 Computes estimated request cost from the configured model-pricing table.
record_usage(model, input_tokens, output_tokens, label)
 Records one priced usage event and enforces a configured hard limit.
total_cost
 Returns accumulated estimated USD cost.
total_input_tokens
 Returns accumulated input tokens.
total_output_tokens
 Returns accumulated output tokens.
warning_reached
 Reports whether accumulated cost reached the warning threshold.
summary()
 Returns aggregate usage and cost values.
reset()
 Clears all recorded events.

backend/services/dictionary_service.py: DictionaryService
load(dictionary_type)
 Loads and cleans the requested entity, attribute, or disease map.
save(dictionary_type, mapping)
 Cleans and writes a complete map.
add_mapping(dictionary_type, raw_term, canonical_term)
 Adds or replaces one mapping and persists it.
remove_mapping(dictionary_type, raw_term)
 Deletes one mapping and persists the result.

clean_mapping(mapping)
 Normalizes keys/values, rejects blanks, and sorts the map.

_get_path(dictionary_type)
 Validates a dictionary type and resolves its configured JSON path.

backend/services/file_manager.py: FileManager

load_settings(settings_path)
 Loads JSON configuration into AppSettings, using defaults when appropriate.

save_settings(settings, settings_path)
 Creates parent directories and writes serialized settings.

load_dictionary(path)
 Loads a JSON mapping.

save_dictionary(dictionary, path)
 Writes a JSON mapping with stable formatting.

load_entity_dictionary()
 Loads the active entity map.

load_attribute_dictionary()
 Loads the active attribute map.

load_disease_dictionary()
 Loads the active disease map.

criteria_to_dataframe(criteria)
 Serializes typed criteria.

save_criteria_csv(criteria, path)
 Writes typed criteria to CSV.

save_criteria_excel(criteria, path)
 Writes typed criteria to Excel.

save_dataframe_csv(df, path)
 Writes any dataframe to CSV after creating its parent directory.

save_dataframe_excel(df, path)
 Writes any dataframe to Excel after creating its parent directory.

load_csv(path)
 Loads CSV into a dataframe.

load_excel(path)
 Loads Excel into a dataframe.

backend/services/codex_bridge.py: CodexBridge

__init__(enabled)
 Stores whether the optional bridge is enabled.

is_available()
 Reports bridge availability.

run_task(task_name, payload)
 Returns a structured disabled/unavailable result or provides the hook for future external task execution.

backend/utils

get_logger(name, log_file)
 Creates or retrieves a consistently formatted logger and optional file handler.

ensure_directories()
Creates configured data, output, log, and dictionary directories.

normalize_whitespace(text)
Collapses whitespace safely.

clean_text(text)
Normalizes whitespace and common text artifacts.

split_sentences(text)
Splits text into sentence-like units.

chunk_text_by_words(text, chunk_size, overlap)
Builds overlapping word chunks while validating overlap constraints.

clean_attribute_key(text)
Produces a normalized dictionary lookup key.

3.5 Integrity validation agent

backend/validation_agent/agent.py

TrialValidationResult.context_dict()
Builds reduced audit context emphasizing incomplete source coverage and weak/unsupported extraction rows.

ValidationRunResult.trial_count
Returns the number of successfully audited trials.

ValidationRunResult.report_context()
Serializes reduced per-trial audit context for grounded follow-up chat.

IntegrityValidationAgent.__init__(llm_settings, cost_monitor, provider)
Injects settings and either an explicit test/provider instance or a factory-created provider.

validate(xml_directory, extraction_file, trial_ids, report_file, stop_event, progress_callback)
Selects trials, aligns workbook rows, supports cooperative cancellation/progress, isolates per-trial errors, detects requested-but-missing IDs, and optionally exports the run.

validate_trial(trial_id, eligibility_text, extracted_rows)
Prompts the auditor, validates its JSON and indices, creates clause/row assessments, supplies conservative defaults for missing row judgments, and calculates metrics.

chat(run, question)
Answers a non-empty follow-up question using completed report context.

_rows_for_prompt(dataframe)
Selects useful columns, converts values to safe strings, and assigns prompt-local row indices.

_parse_json_object(text)
Parses plain or fenced JSON and recovers the outermost object when necessary.

_is_valid_index(value, row_count)
Checks whether a model-returned index is an integer within bounds.

_valid_indices(values, row_count)
Filters, deduplicates, and sorts model-returned indices.

_calculate_metrics(trial_id, clauses, extractions, rows)
Calculates weighted estimated precision/recall/F1 and coverage, unsupported, missing-value, and atomicity rates.

save_report(run, path)
Writes status, summary, source coverage, extraction support, quality notes, and errors to separate Excel sheets.

backend/validation_agent/prompts.py

build_validation_prompt(trial_id, eligibility_text, extracted_rows)

Requests a JSON audit of source-clause coverage and extraction support, with explicit evidence and confidence fields.

build_chat_prompt(report_context, question)

Constrains follow-up answers to completed audit evidence and includes the user's question.

3.6 Tkinter presentation layer

frontend/app.py: ModernAutoCritApp

__init__()

Creates the root window, shared state, visual style, and application tabs.

_configure_style()

Defines Tkinter theme, fonts, colors, and widget styles.

_build_ui()

Creates navigation/header content and mounts Run, Settings, Dictionary, Analytics, and Integrity views.

launch_app()

Constructs the application and enters the Tk event loop.

frontend/views/run_view.py

QueueLogHandler.__init__(log_queue)

Connects a logging handler to a thread-safe message queue.

QueueLogHandler.emit(record)

Formats a log record and queues it for UI consumption.

RunView.__init__(parent, state)

Initializes selection, progress, queue, and worker state and builds the view.

_build_ui()

Constructs paths, controls, status, summaries, progress, and collapsible log widgets.

_configure_logging()

Attaches the queue handler to relevant loggers.

select_xml_folder() / select_output_file()

Opens selection dialogs and updates bound paths.

_count_xml_files(xml_directory)

Counts input files for progress estimation.

start_pipeline()

Validates inputs, resets state, and starts background extraction.

_run_pipeline_worker(xml_directory, output_file)

Runs the route in a worker thread and queues success or failure.

_poll_queues()

Schedules recurring result/log polling on the Tk thread.

_poll_log_queue()

Moves queued log messages into the UI.

_update_progress_from_log(message)

Infers completed-trial progress from pipeline log messages.

_poll_result_queue()

Dispatches queued worker completion to success/error handlers.

```

_handle_success(summary)
    Restores controls, presents counts/cost/output, and updates shared state.
_handle_error(error_message)
    Restores controls and presents a failure.
toggle_logs()
    Shows or hides log details.
append_log(message)
    Appends one message to the log widget.
clear_log()
    Clears displayed logs.
destroy()
    Detaches logging resources before widget destruction.

frontend/views/settings_view.py: SettingsView
__init__(parent, state)
    Creates settings variables, provider-test queues, and the form.
_build_ui()
    Builds provider/model, credential, extraction, cost, and directory controls.
_on_provider_changed(event)
    Refreshes model/provider-specific controls after selection.
_update_provider_fields()
    Applies provider-specific labels, defaults, and visibility.
_set_availability_not_tested()
    Resets the connection-test indicator.
_build_test_settings()
    Constructs validated temporary LLM settings from current form values.
test_provider()
    Starts a non-blocking provider connection test.
_test_provider_worker(test_settings)
    Creates/tests the provider off the Tk thread and queues the result.
_poll_test_results()
    Moves test completion back onto the UI thread.
save()
    Validates form values, updates shared settings/cost state, and persists configuration.

frontend/views/dictionary_view.py: DictionaryView
__init__(parent)
    Initializes mapping selection, filtering, and editor state.
_build_ui()
    Builds dictionary selector, search, table, and edit controls.
refresh()
    Reloads the selected dictionary and refreshes the table.
populate_table()
    Filters and renders raw/canonical pairs.
_on_row_selected(event)
    Copies the selected mapping into editor fields.

```

add_or_update_mapping()
Validates and persists an edited mapping.

delete_selected()
Confirms and removes the selected mapping.

clear_fields()
Clears editor selection and inputs.

frontend/views/analytics_view.py: AnalyticsView

__init__(parent, state)
Initializes summary variables, tables, paths, and UI.

_build_ui()
Builds output selection, summary cards, criteria/unmapped/anomaly tables, and open-file controls.

_create_summary_card(parent, column, label, variable)
Creates one reusable metric card.

_create_table(parent)
Creates a scrollable dataframe preview table.

select_output_file()
Selects a criteria workbook and refreshes analytics.

refresh()
Loads output and companion workbooks, populates tables, and recalculates summaries.

_resolve_output_path()
Chooses the explicitly selected or most recent pipeline output.

_get_unmapped_path(output_path) / _get_anomalies_path(output_path)
Derives companion workbook paths.

_refresh_summary_from_state(output_df)
Uses the last pipeline summary where available and dataframe fallbacks otherwise.

_clear_anomaly_summary()
Nested refresh helper that zeros anomaly cards when no report is available.

_load_dataframe_into_tree(df, tree)
Replaces table columns and rows from a dataframe.

open_output_file() / open_unmapped_file() / open_anomalies_file()
Validates and opens the corresponding workbook.

_open_path(path)
Delegates file opening to the operating system.

frontend/views/integrity_view.py: IntegrityView

__init__(parent, state)
Initializes path, selection, event-queue, cancellation, report, and chat state.

_build_ui()
Builds audit inputs, controls, progress/summary area, and grounded chat panel.

_path_row(parent, row, label, variable, command)
Creates a reusable labeled path selector.

_select_xml_directory() / _select_extraction_file() / _select_report_file()
Opens the relevant selection dialog.

_start_validation()
Validates inputs and starts the nested background `work()` function, which runs the agent and

posts events.

`_stop_validation()`
Sets cooperative cancellation so no additional trial begins after the active request.

`_start_chat()`
Validates chat state and starts a nested `work()` function for the model request.

`_send_from_event(event)`
Maps the keyboard event to chat submission.

`_poll_events()`
Processes progress, audit, error, and chat events safely on the Tk thread.

`_show_validation_summary(run)`
Renders aggregate run results and report location.

`_append_chat(speaker, message)`
Adds a labeled message to the chat transcript.

4 Command-line data utilities

4.1 `data_download.py`

`_api_get(path, ...)`
Calls the ClinicalTrials.gov v2 API with timeout, session, and response/error handling.

`_add_text(parent, tag, value)`
Adds a non-empty XML child element.

`study_to_xml(study)`
Transforms a v2 API study object into the XML shape expected by the pipeline.

`save_study_xml(study, output_dir)`
Serializes one transformed study under its NCT identifier.

`download_trial_xml(nct_id, output_dir, sleep_seconds, session)`
Fetches and saves one specified trial.

`search_studies(search_terms, number, ...)`
Paginates search results to the requested limit and warns about server-reported truncation.

`download_search_results(search_terms, number, output_dir, ...)`
Searches, converts, and saves a batch of trials.

`download_xml_from_dataframe(df, nct_column, output_dir, max_trials)`
Downloads trials listed in a dataframe column.

`build_parser()`
Defines CLI arguments and help.

`_confirm_large_download(number, threshold)`
Requests interactive confirmation above the safety threshold.

`main(argv)`
Validates CLI input, enforces large-download confirmation, runs the download, and returns a process status.

4.2 `dict_mapping.py`

`_text(value)`
Converts non-missing cell content into normalized text.

`_first_text(row, *columns)`
Returns the first usable value among candidate column names.

_add_if_new(mapping, raw, canonical)
 Adds a mapping only when its normalized raw key is absent.

update_dictionaries(output_file)
 Reads an extraction workbook, routes terms to entity/attribute/disease dictionaries, preserves existing decisions, saves changes, and returns counts.

reset_dictionaries()
 Restores all active maps from their original snapshots.

build_parser()
 Defines update/reset command-line options.

main(argv)
 Dispatches reset or update behavior and returns a process status.

5 Tests, benchmarks, backups, and placeholders

5.1 Test functions

tests/test_data_download.py

FakeResponse.__init__ / raise_for_status / json
 Provide a deterministic requests-like response for downloader tests.

FakeSession.__init__ / get
 Queue fake responses and record API requests.

test_download_search_results_writes_pipeline_compatible_xml
 Verifies batch download output can be consumed by the XML service.

test_search_studies_uses_pagination
 Verifies page-token traversal and requested limits.

test_api_limit_has_clear_warning
 Verifies a clear warning for API-limited result sets.

test_large_noninteractive_download_requires_yes
 Verifies the large-download guard; its nested `fake_download()` prevents network access.

tests/test_dict_mapping.py

test_update_routes_terms_and_preserves_existing_mapping
 Verifies routing, additions, and non-overwrite behavior.

test_reset_restores_all_snapshots
 Verifies all original dictionaries are restored.

test_missing_required_columns_is_clear
 Verifies a useful error for incompatible workbooks.

tests/test_integrity_validation.py

FakeProvider.__init__ / default_model / generate
 Provide deterministic structured LLM responses.

_write_trial(path)
 Creates a minimal source trial fixture.

test_validation_computes_metrics_and_writes_report
 Checks classification, metrics, and workbook sheets.

test_missing_requested_trial_is_reported
 Checks missing NCT-ID error reporting.

test_unreturned_extraction_gets_a_conservative_assessment

Checks the weak-support fallback for omitted rows.

test_validation_can_be_stopped_before_next_trial

Checks cooperative cancellation between trials.

The four `tests/benchmark*.py` files are currently one-line/empty scaffolds and define no functions. They reserve locations for model, metric, semantic, and overall benchmarking.

5.2 Retained PySide backup

The `frontend_pyside_backup` tree is not the active UI. It preserves the earlier implementation for reference:

- `app.py`: `ModernAutoCritApp.__init__()` and `launch_app()` create and run the Qt application.
- `components/dictionary_editor.py`: constructor, `refresh()`, and `add_mapping()` implement the earlier dictionary editor.
- `components/file_table.py`: constructor, `load_dataframe()`, and `clear_table()` render tabular results.
- `views/analytics_view.py`: constructor, `refresh()`, `populate_table()`, and `open_output()` implement the earlier analytics page.
- `views/run_view.py`: `QtLogHandler` constructor/`emit()`, view constructor, `_build_ui()`, `_configure_logging`, `select_xml_folder()`, `select_output_file()`, `start_pipeline()`, `pipeline_finished()`, `pipeline_failed()`, `_set_idle_state()`, `_clear_worker_references()`, `append_log()`, `clear_log()`, and `closeEvent()` implement threaded Qt execution.
- `views/settings_view.py`: constructor, `update_model_options()`, and `save()` implement earlier settings management.
- `workers/pipeline_worker.py`: constructor and `run()` wrap the pipeline in a Qt worker with signals.

5.3 Empty compatibility and future-extension modules

The following contain no functions: package `__init__.py` markers; `backend/services/dictionary_services.py`; `backend/quality/metrics.py`; `backend/quality/validation_agent.py`; and the consistency/unit rule modules. Their presence should not be mistaken for implemented behavior. The singular `dictionary_service.py`, dedicated `backend/validation_agent` package, and numeric/terminology rule files contain the current implementations.

6 What changed from AutoCriteria

Concern	Upstream AutoCriteria	Modern AutoCrit
Code organization	A long extraction script combines prompts, LangChain invocation, retries, XML iteration, cleanup, and dataframe mutation [1].	Modules and typed collaborators isolate core extraction, schemas, services, quality rules, validation, UI, and utilities.

Concern	Upstream AutoCriteria	Modern AutoCrit
Model integration	LangChain <code>PromptTemplate</code> , <code>PipelinePromptTemplate</code> , <code>ChatOpenAI</code> , and <code>LLMChain</code> target GPT-4 [1].	A provider contract and factory wrap the modern OpenAI Responses SDK; token usage and cost are first-class. OpenAI is implemented today, while other provider labels remain extension points.
Execution	A command-line Python script processes files and writes output [1].	Backend CLI plus a standalone Tkinter desktop application with configuration, progress, logs, dictionary editing, analytics, and integrity auditing.
Configuration	API key and major behavior are embedded close to script execution.	Pydantic settings and JSON persistence separate provider, extraction, cost, directory, and dictionary configuration.
Prompting/parsing	Large inclusion/exclusion templates and LangChain output parsing live beside control flow.	Prompt builders and defensive JSON parsing are separate, testable domain modules.
Normalization	Extensive procedural cleanup occurs while appending dataframe rows.	A dedicated normalizer applies deterministic maps, produces unmapped-term reports, and separates exact/fuzzy duplicate analysis.
Quality assurance	The upstream script focuses on extraction and cleanup.	Deterministic anomaly rules plus an independent LLM-assisted audit examine plausibility, terminology, source coverage, and unsupported rows.
Data acquisition	The upstream repository directs users to prepare downloaded XML [1].	A ClinicalTrials.gov v2 downloader searches, paginates, confirms large batches, and emits pipeline-compatible XML.
Usability	Research-oriented command-line workflow.	Standalone desktop workflow, direct workbook access, persistent settings, and operational feedback; an in-browser option is planned.
Extensibility	Changing a concern generally edits the central script.	Interfaces and layer boundaries allow providers, rules, views, dictionaries, validators, and exporters to evolve separately.

Modern AutoCrit is a reimplementaion and architectural evolution, not evidence that the upstream scientific results have automatically transferred. Any comparative performance claim requires a shared dataset, annotation protocol, model/version controls, and measured evaluation.

7 Key milestones

1. **Repository decomposition.** Established backend, frontend, configuration, data, documentation, output, and test boundaries, then moved working extraction behavior into `core/services/schemas`

rather than one long script.

2. **Modern SDK migration.** Replaced LangChain-dependent execution with direct modern OpenAI SDK usage and introduced a provider-neutral interface, response contract, factory, retries, usage accounting, and cost monitoring.
3. **End-to-end pipeline.** Added XML ingestion, inclusion/exclusion splitting, chunked extraction, typed parsing, normalization, deduplication, unmapped-term reporting, and Excel export.
4. **Standalone application.** First delivered a PySide GUI, then moved the active desktop interface to Tkinter for stability while retaining the earlier implementation as a backup.
5. **Operational tooling.** Added settings persistence, live logs, background execution, progress reporting, dictionary management, analytics, downloader/update CLIs, and benchmark scaffolds.
6. **Quality layer.** Added deterministic anomaly detection for numeric, unit, terminology, and missing-value problems.
7. **Integrity agent.** Added an independent source-versus-output audit, estimated coverage/support metrics, multi-sheet reports, cancellation/progress, and evidence-grounded follow-up chat.
8. **Dictionary expansion.** Grew active terminology maps, retained original snapshots, and added safe append/reset workflows.

8 Current limitations and technical debt

- Only the OpenAI provider implementation is present even though settings and factory branches anticipate Anthropic, Gemini, and Ollama.
- Integrity metrics are generated from an LLM audit and should be labeled estimated until calibrated against clinician-reviewed annotations.
- Inclusion/exclusion heading detection is intentionally conservative and can fail on irregular registry formatting.
- Exact deduplication is active; fuzzy duplicate discovery exists but is advisory, and the settings flag for semantic deduplication does not itself constitute a complete embedding pipeline.
- Some placeholder/legacy modules and the older OpenAI client remain, creating ambiguity that should be resolved as interfaces stabilize.
- The desktop progress bar partly infers trial completion from logs rather than receiving structured pipeline progress events.
- Dictionary update suggestions currently depend on observed workbook terms and deterministic matching, not ontology identifiers or embedding confidence.
- Clinical terminology, thresholds, and generated findings require expert governance, versioning, provenance, and reproducible evaluation before clinical use.

9 Future work: toward an agentic eligibility-extraction system

The future goal is a supervised agentic system in which specialized components propose, verify, criticize, and improve clinical-trial eligibility extraction while preserving source evidence and human authority.

9.1 Terminology grounding with RxNorm and ICD-10

Research and, where licensing and fit permit, integrate established terminology sources. RxNorm can support normalized medications and ingredients; ICD-10 can support disease/category coding. The implementation should store source vocabulary, code, display term, version, match method, and

confidence rather than replacing text with an unexplained label. Synonym resolution should retain the original phrase and source sentence, and ambiguous matches should be review tasks rather than silent substitutions.

9.2 Embedding-assisted adaptive dictionaries

Add an embedding retrieval layer over reviewed canonical mappings. For an unseen term, retrieve candidates, combine semantic similarity with lexical/contextual evidence, and emit a ranked suggestion with confidence and provenance. A reviewer accepts, edits, or rejects the proposal. Accepted mappings enter a versioned dictionary and a regression set; automatic mutation of production dictionaries should be avoided until thresholds are empirically validated.

9.3 Reviewer/critic agent

Introduce a critic that receives source text, extracted rows, the validation agent’s claims, and explicit evaluation criteria. It should test evidence alignment, index correctness, internal consistency, missed clauses, unsupported assertions, metric arithmetic, and uncertainty calibration. To reduce correlated errors, the critic should use independently phrased prompts and, when practical, a different model/provider. Disagreements should be escalated to human review and captured as evaluation data rather than decided by model confidence alone.

9.4 Browser-based deployment

Extract a stable application service/API boundary and add an optional browser UI while retaining the desktop client. Web deployment requires authentication, encrypted secret handling, job queues, cancellation, artifact storage, audit logs, concurrency/cost controls, and a clear policy that prohibits protected health information unless an appropriately secured deployment is established.

9.5 Proposed agentic loop

1. An ingestion agent validates registry provenance and prepares trial text.
2. A segmentation agent identifies inclusion/exclusion structure and atomic clauses.
3. An extraction agent produces typed, source-grounded rows.
4. A terminology agent proposes ontology and dictionary normalization with provenance.
5. Deterministic rules screen numeric, unit, schema, and consistency failures.
6. The current integrity agent audits source coverage and extraction support.
7. A reviewer/critic agent challenges both extraction and validation findings.
8. A resolution stage routes disagreements and low-confidence items to a human reviewer.
9. Approved corrections update evaluation fixtures and, through controlled review, adaptive dictionaries.

Every agent transition should use a versioned schema, retain the exact source span, record model/prompt/vocabulary versions and cost, and be replayable. Release gates should be based on held-out, clinician-reviewed precision/recall and safety metrics, not model self-evaluation. The system should assist extraction and review; final interpretations of patient eligibility remain with qualified trial personnel.

10 Recommended evaluation program

- Build a stratified, clinician-annotated corpus across disease areas, phases, criteria formats, and numerical/temporal complexity.
- Measure clause segmentation, entity/attribute/value extraction, temporality, ontology grounding, atomicity, hallucination rate, and full-source coverage separately.
- Compare the modern pipeline with the reproducible upstream baseline under controlled model availability, noting that original GPT-4 behavior may not be exactly reproducible.
- Calibrate validator and critic confidence against human adjudication; measure whether the critic improves error detection rather than merely increasing disagreement.
- Maintain adversarial cases for prompt injection in trial text, malformed XML/workbooks, ambiguous terminology, unit traps, negation, exceptions, and nested criteria.
- Version datasets, prompts, models, dictionaries, ontology releases, and metrics so changes are attributable.

References

- [1] S. Datta et al., “AutoCriteria source repository.” <https://github.com/surdatta/autocriteria/tree/main>
- [2] S. Datta et al., “AutoCriteria: a generalizable clinical trial eligibility criteria extraction system powered by large language models,” Journal of the American Medical Informatics Association. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10797270/>